Recurrent Energy Wave Transformer
Learning a Joint Fourier Energy Landscape

Kenju Tomita · kt7432@g.rit.edu
Rochester Institute of Technology
CSCI 635: Introduction to Machine Learning
Individual Project Proposal · October 9, 2026

Abstract

I propose the **Recurrent Energy Wave Transformer (REWT)**, a fully energy-based language model that learns a Fourier description of its energy landscape. One energy function will score all internal states and candidate predictions together. Training will learn the wave components; inference will repeatedly revise states and predictions with trained weights fixed. REWT will combine my previous attention-energy, diffusion, forecasting, and specialist mechanisms. I will implement it from scratch and compare it with attention and non-Fourier energy models across three random seeds. Experiments will assess generated text, reasoning, prediction, stability, and computational cost.

1. Introduction and Research Objectives

Language models predict what comes next from observed text. Transformers use attention to connect relevant information [1]; energy-based models score how well states or predictions fit their context [2,3]. REWT will express these relationships through one learned energy. Here, **energy** is a numerical score, and **wave** means Fourier sine/cosine components. The **energy landscape** describes how the score changes as internal states and candidate predictions change.

A **latent state** is an internal representation. REWT will learn Fourier components covering the joint energy over all states and outputs. A finite representation will capture selected structure, without guaranteeing an exact reconstruction or global minimum. **Training changes the parameters that define the landscape; inference follows its gradients to change states and predictions.** The training loss over weights is a separate objective.

The research question is: **Does a learned Fourier energy landscape improve text generation and reasoning enough to justify its computational cost?** Fourier features have helped low-dimensional function fitting [10]; their usefulness here remains unproven. My contribution will be the combined architecture and evaluation, crediting established methods. Efficient state adaptation could benefit language-processing systems with limited computing resources.

Prior results, completed before this project. A three-seed CPU study of approximately 145,000-parameter models reached **2.5121 test bits per byte** for my energy/local-attention hybrid, versus **2.5782** for full attention and **2.5770** for local attention. Lower is better. Models used 128-byte context and equal 90-second training-update budgets, excluding setup and evaluation [5]. A historical 10.6-million-parameter study instead favored full attention: **1.6633 versus 1.7240 validation bits per byte** for pure energy after 40.96 million training bytes [6].

Combining mechanisms has not consistently helped. A three-seed, 24-update pilot reached **2.4265 test bits per byte**, versus **2.4247** for ordinary continuation [7]. A synthetic decision study reached **76.95% accuracy** with an energy refiner, versus **72.66%** with its combined variant [8]. These precursor experiments motivate comparisons; they do not establish fully energy-based REWT performance.

2. Proposed Model and Learning Process

Fully energy-based architecture. I will independently implement REWT in PyTorch. For each next-byte prediction, recurrent states, attention-related states, specialist workspaces, and output logits will form one joint state. Embeddings and proposal networks will provide observed context and initial guesses. Every block and output will then be updated through the same scalar energy.

Learning the joint Fourier landscape. I will learn one energy over the entire joint state:

$$E_\theta(h_t, s) = Q_\theta(h_t, s) + \Sigma_{k=1}^{K} a_k(h_t) \cos(\omega_k^\top s + \varphi_k)$$

Here, t is the last observed byte position; h_t represents its observed context; s collects all internal states and output logits; and θ denotes learned parameters, including Fourier parameters. $E_\theta(h_t, s)$ is one numerical score; lower is preferred. $Q_\theta(h_t, s)$ is a positive quadratic penalty pulling states toward context-informed values.

K is the number of wave components (modes); k labels each one; and $\Sigma_{k=1}^{K}$ adds them. $a_k(h_t)$ is a learned, context-dependent strength multiplying the cosine wave cos. ω_k sets its direction and rate of change across state coordinates. $\top$ means transpose, so $\omega_k^\top s$ is a dot product: a weighted sum of those coordinates. φ_k shifts the wave. Shared modes couple blocks; I will start with K = 16 and constrain frequency magnitudes.

Minimization during inference. Each block will update its state using the partial gradient of the same total energy. Jointly, the update is:

$$s^{(r+1)} = s^{(r)} - \eta \nabla_{s^{(r)}} E_\theta(h_t, s^{(r)})$$

Here, r counts refinement rounds; parentheses in $s^{(r)}$ mark a round, not a power. $s^{(r)}$ is the current joint state and $s^{(r+1)}$ the next one. η is a positive step size. $\nabla_{s^{(r)}} E_\theta$ is the vector of energy derivatives with respect to the current state. Subtracting the scaled gradient moves toward lower energy. h_t and θ stay fixed during these rounds.

In plain language: **guess, score, revise, and repeat.** Refined states carry information across observed tokens; softmax converts output logits into probabilities. Backtracking accepts steps only if they reduce the full energy. Context coefficients can be cached and derivatives evaluated directly; every evaluation counts toward cost.

Spectral scale and noise. Coarse-to-fine gradient proposals will be compared with all-mode updates and checked against the complete energy. Separately, filtering past training gradients will test whether spikes decrease without quality loss. Validation will assess whether damping removes useful information. Inspired by FNet's Fourier token mixing [9], transformed past states may also provide context.

Combining previous mechanisms. Diffusion, forecasts, recurrent/feedforward proposals, specialists, and critic/routing mechanisms will initialize candidates or parameterize energy compatibility. All candidates will undergo minimization. Forecasts and output candidates will remain separate from observed input.

Training. All parameters, including Fourier parameters, will start randomly. Staged then joint training will differentiate through two refinement rounds. Next-byte cross-entropy will penalize incorrect predictions; an energy-ranking loss will favor correct candidates over sampled alternatives. Training-only forecasts will use frozen course-trained targets; diffusion will learn denoising. AdamW will update weights; validation will select learning rates and auxiliary-loss settings. Inference will keep weights fixed.

3. Evaluation, Feasibility, and Timeline

Data and controls. enwik8 [4] will use 256 byte values, 90M/5M/5M-byte training/validation/test splits, and 128-byte context. Standard Transformer, non-Fourier energy, basic REWT, and REWT with support modules will target 250,000 parameters (within 0.5% where feasible) and three seeds: up to 12 model/seed settings. Energy controls share states, priors, and conditioning; data, batches, optimizer, and tuning budgets will match.

Generated text quality. Use 30 fixed, held-out 32-byte prefixes and 96-byte continuations. Match decoding settings and sampling seeds. Rate anonymized outputs from 1–5 for fluency, coherence, and relevance using a fixed rubric; randomize presentation order. Measure repeated-phrase rates. Save all outputs and ratings; show examples selected by fixed prompt IDs.

Reasoning quality. Fine-tune separate copies of every model on identical short arithmetic and rule-chain tasks. Test disjoint instances and withheld templates/chains; score exact final answers and complete, valid intermediate steps with deterministic checkers. Prompt plus answer will fit 128 bytes. Report accuracy by step count. These tasks measure bounded reasoning skills.

Prediction, confidence, and cost. Record bits per byte (lower is better), distractor/delay recall accuracy, and Brier score (confidence versus outcomes). Report means and variation across seeds, and quality versus training time, latency, gradient calls, and memory. Measure gradient norms and spikes with thresholds fixed before main runs. Lower energy alone will not establish better outputs.

Controls and component tests. Select settings on validation; reset states at matched window boundaries; verify past-only inputs, serial/chunked consistency, and refinement gradients. Compare learned/fixed frequencies, coupled/separate modes, and full-band/coarse-to-fine refinement. Filtering controls include none, clipping, and smoothing; remove individual modules in smaller tests. Sweep 0/1/2/4/8 rounds, varying state/output updates separately. Match size/data and separately training time; record unstable runs.

Feasibility and individual work plan. Core models and text/reasoning tests come first. The 12-GPU-hour plan allocates two to verification/profiling, eight to training including reasoning adaptation, and two to evaluation; all auxiliary costs count. Profiling sets common budgets with a 20% margin. Reduce size/bytes if needed, retaining three seeds. Extra modules/ablations use remaining time. I will implement, analyze, and write, with peer proofreading; course code and weights will be new.

Dates	Milestone and output
Oct. 9	Submit the proposal.
Oct. 10–16	Build the joint Fourier model; present the proposal on Oct. 16.
Oct. 17–Nov. 1	Integrate support modules and implement gradient filtering.
Nov. 2–15	Train comparisons and reasoning copies; prepare blinded text ratings.
Nov. 16–29	Score text, reasoning, recall, and costs; finish held-out testing.
Nov. 30–Dec. 8	Analyze results, prepare figures, and obtain peer feedback.
Dec. 9	Submit the final report and code/artifacts.
Dec. 10–13	Prepare the final presentation; deliver it on Dec. 13.

References

1. Vaswani, A., et al. “Attention Is All You Need.” *NeurIPS*, 2017. Conference proceedings.

2. Gladstone, A., et al. “Energy-Based Transformers Are Scalable Learners and Thinkers.” *ICLR*, 2026. Conference proceedings.

3. Hoover, B., et al. “Energy Transformer.” *NeurIPS*, 2023. Conference proceedings.

4. Mahoney, M. “About the Test Data,” Large Text Compression Benchmark. Dataset source and checksums.

5. EnergyTransformer project. “Compute-Controlled Confirmation.” Prior local report, September 2026. Local report.

6. EnergyTransformer project. “Reconciled Evidence and Repair Results.” Prior historical-results audit, September 2026. Local report.

7. EnergyTransformer project. “Native Real-Text Pilot: Negative Result.” Prior experimental-stack study, September 2026. Local report.

8. EnergyTransformer project. “Calibration and Reproducibility Follow-up.” Prior synthetic decision-adapter study, September 2026. Local report.

9. Lee-Thorp, J., et al. “FNet: Mixing Tokens with Fourier Transforms.” *NAACL*, 2022. Conference proceedings.

10. Tancik, M., et al. “Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains.” *NeurIPS*, 2020. Conference proceedings.